# Fusion 360 MCP

**Model parts and assemblies in Autodesk Fusion 360 by talking to Claude.**

You describe what you want — "a 60×40×20 mm enclosure, 2 mm walls, four M3 tapped holes in the corners" — and Claude builds it in Fusion, looks at the result with a screenshot, checks the dimensions, and fixes its own mistakes. It can also export STEP/STL/DXF and save versioned designs.

This is a hobbyist project shared as-is. It works well for the bread-and-butter of mechanical CAD (boxes, cylinders, holes, fillets, shells, tapped holes, multi-part assemblies with joints) and hands off to raw Fusion API code for the exotic stuff.

## What you get

About 42 tools Claude can call, grouped as:

- **Primitives & features** — boxes, cylinders, holes, fillets, shells, tapped holes (with correct tap-drill sizing for hand-tapping). Cutting tools auto-report volume change and warn on no-op cuts.
- **Parametric** — named user parameters so models stay adjustable.
- **Faces** — sketch and drill on a selected face, not just the base plane.
- **Assemblies** — components, occurrences, linear copies of hardware, grounding, and rigid / revolute joints.
- **Verification & diagnostics** — return a screenshot to Claude, measure real distances, ASCII cross-sections (solid/void scans), face/edge listings with stable IDs, and a rich one-call model-state report.
- **Async & resilience** — long operations run as pollable jobs with incremental progress; liveness/state stay responsive even during a hung op; the listener refuses known host-crashing API calls.
- **Document safety** — every state read shows the active document name + lineage; saves report which lineage they wrote; save-as-new and read-only version listing guard against overwriting the wrong design.
- **Export & save** — STEP, STL (3D print), DXF (laser/waterjet), and save / open / list designs across sessions.
- **Escape hatch** — `run_fusion_code` / `run_fusion_code_safe` run arbitrary Fusion API Python (with injected unit/observability helpers and documented sketch-plane mapping), the latter with automatic rollback on error.

Full list: `docs/TOOLS.md`.

## How it works (the 30-second version)




Two long-running pieces:

1. **MCPListener** — a small add-in that runs *inside Fusion* and executes modeling commands on Fusion's main thread.
2. **fusion_mcp_server.py** — an MCP server that **Claude Desktop launches for you** in the background; it relays Claude's tool calls to the listener.

You don't start the server yourself — Claude Desktop does, based on its config. You just keep Fusion open with the add-in running.

---

## Quick start

1. **Install** — follow `docs/INSTALL.md`. It covers the add-in, the Python deps, and the Claude Desktop config, with fixes for the common snags.
2. **Check your setup** — run the preflight: - Windows: double-click `launch.bat` - macOS/Linux: `./launch.sh` It tells you exactly what's missing.
3. **Confirm it's live** — in Claude Desktop, ask: **"Ping Fusion."** A reply of "alive" means everything's connected.
4. **Build something** — try:

> "Make a 20 mm cube and show me."

then the real shakedown:

> "Make a 60×40×20 mm parametric box, shell it to 2 mm with an open top, add an M3 tapped hole 5 mm in from each top corner, and show me top and iso views."

---

## Requirements

- Autodesk Fusion 360 (current version).
- Claude Desktop (the desktop app — this uses local MCP over stdio).
- Python 3.10+ (3.10–3.12 are the safest; see INSTALL note on 3.13/3.14).

## Files in this package

| File | What it is |
|---|---|
| `MCPListener.py` | The add-in that runs **inside Fusion**. |
| `fusion_mcp_server.py` | The MCP server **Claude Desktop launches**. |
| `launch.py` / `launch.sh` / `launch.bat` | Preflight health-checker. |
| `docs/claude_project_instructions.md` | Paste into a Claude Project to teach it the conventions. |
| `requirements.txt` | Python dependencies. |
| `docs/INSTALL.md` | Step-by-step install + troubleshooting. |
| `docs/TOOLS.md` | Reference for every tool. |
| `docs/ARCHITECTURE.md` | How the pieces fit, for the curious / for hacking. |
| `LICENSE` | MIT. |

## A few honest caveats

- The listener executes arbitrary Python sent to **localhost only** ( `127.0.0.1:8765` ). That's required for it to work, but don't expose that port to a network. See ARCHITECTURE for the security model.
- The canned tools assume principal planes and box-like geometry for face selection. Complex/organic shapes use the raw-code escape hatch — Claude knows when to switch.
- This isn't an official Autodesk or Anthropic product. It's glue code. Treat early runs as a shakedown and keep your work saved.

Happy modeling.

# Changelog

## 1.2.1

Fixes from a project-wide review triggered by a real degraded-state incident (old listener + new server = every payload empty while status read OK).

- **CRITICAL: switched to ThreadingHTTPServer.** The v1.2.0 listener used a single-threaded HTTPServer, so while `/exec_sync` held the one HTTP thread waiting on a job (up to ~110s), `/ping` and `/state` could not answer - silently defeating the "separate always-responsive channel" design. Verified fixed: ping now answers instantly during a blocked sync call.
- **Version/protocol handshake.** The listener now reports `listener_version` + `protocol` in every response; the server checks it and returns a loud PROTOCOL MISMATCH message with exact fix steps when the add-in is stale - instead of silently rendering empty payloads as STATUS: OK. This makes the incident that prompted this release impossible to miss.
- **Fail-loud capture health.** Envelopes include `stdout_bytes`; the server marks ok-but-silent results with "(no output captured)".
- **Job-store pruning.** Finished jobs are capped at the 50 most recent so long sessions don't grow memory unboundedly.
- **Docs:** INSTALL now warns that Fusion loads the add-in from its AddIns folder, not from wherever the project files live - updating one does not update the other.

## 1.2.0

Async execution and document safety — driven by a second round of real-use feedback. The core fix: a 4-minute timeout that told you nothing about what actually happened in Fusion. Execution is now decoupled from the HTTP response.

### Added — async job model (the big one)

- **Listener is now async.** `POST /exec` enqueues a job and returns a `job_id` immediately; the operation runs on the main thread and is polled via `GET /job/<id>`, which returns a structured envelope (`status`, `stdout`, `traceback`, `elapsed_ms`) and **partial stdout while still running** (incremental progress).
- **`run_async(code)` + `job_status(job_id)`** tools for long operations that shouldn't block. Plus a `progress(msg)` helper to mark stages visible mid-run.

- **Ping/state on a separate channel.** `GET /ping` and `GET /state` are served from cache without the main thread, so they STAY RESPONSIVE during (or after) a hung operation. A hung op and a dead listener are now distinguishable.
- `get_state()` — cheap, guaranteed-fast document identity + counts (no geometry work). The safe first call for crash recovery.
- **Structured result envelope, always.** Every execution returns status/stdout/traceback/elapsed; no more bare "no result received". The sync path returns partial output and a job id if the op outlives the wait.
- **Watchdog** flags jobs that exceed their budget as `timeout` (clear signal rather than a dead channel).

### Added — document safety

- **Document identity surfaced everywhere.** `get_state` and `get_model_state` lead with the active document **name + lineage id**, preventing wrong-document overwrites.
- `save_document(name, create_new=False)` now reports **which lineage it versioned**, and `create_new=True` saves a brand-new lineage so a new part never buries an unrelated design's history.
- `list_versions(design_id="")` — read-only enumeration of a lineage's saved versions, so a bad save is recoverable and the user can be pointed to the exact version to promote.
- `open_document` reports the new active lineage.

### Added — documentation

- Sketch-plane / axis mapping (XY/XZ/YZ + offset planes, incl. the XZ→world-−Z inversion) documented in `run_fusion_code`'s docstring.

### Notes

- `run_fusion_code` uses the bounded sync path (~100s) then hands off to polling; no behavior change for short ops.
- Volume-delta no-op detection and the `mm()` unit helper shipped in 1.1.0 and already cover two earlier feedback items.

## 1.1.0

Observability, diagnostics, and stability. Injected unit/observability helpers (`mm()`, `assert_scale()`, `report_volume_delta()`, etc.); auto volume-delta reporting with no-op warnings on cut/shell tools; `cross_section()`, `list_faces()`, `list_edges()`, `heartbeat()`; listener-level refusal of known host-crashing calls; richer `get_model_state()`.

## 1.0.0

First public release. Fusion 360 add-in listener + MCP server with ~30 tools (primitives, parameters, faces, assemblies/joints, measurement, visual feedback, export, persistence), preflight launcher, and full docs.

# Install guide

Three parts to install, then a verification step. Budget ~15 minutes the first time. Read the **Common snags** at the bottom if anything misbehaves — every issue listed there is one a real user actually hit.

```
Part A — the Fusion add-in (the listener)
Part B — Python + dependencies
Part C — register with Claude Desktop
Verify  — preflight check, then ping from Claude
```

## Part A — Install the Fusion add-in

The listener must be an **Add-In**, not a Script. (Scripts run once and exit; add-ins stay loaded — which the listener needs.) This trips people up, so follow the type carefully.

1. Open Fusion 360.
2. Press **Shift+S** (or **Utilities → Add-Ins → Scripts and Add-Ins**).
3. Click the **Add-Ins** tab (not Scripts).
4. Click the green **+** to create. In the dialog choose **Type: Add-In**, **Language: Python**, **Name:** `MCPListener`, then Create. - This creates a folder at:
   - **Windows:** `%appdata%\Autodesk\Autodesk Fusion 360\API\AddIns\MCPListener\`
   - **macOS:** `~/Library/Application Support/Autodesk/Autodesk Fusion 360/API/AddIns/MCPListener/`
5. Select `MCPListener` in the list, click the **pencil / Edit** to open its folder, and **replace the generated** `MCPListener.py` with the one from this package (same filename).
6. Back in the Add-Ins tab, select `MCPListener`, tick **Run on Startup**, and click **Run** (▶). A popup should say *"MCPListener running on http://127.0.0.1:8765"*.

> **The AddIns folder is the only copy Fusion runs.** If you keep the project files elsewhere (a repo/working folder), updating that copy does NOT update the add-in - you must copy the new `MCPListener.py` into the AddIns folder above and restart the add-in (Stop › Run). A stale add-in with a newer server now reports PROTOCOL MISMATCH instead of failing silently.
>
> Leave Fusion open while using Claude. With "Run on Startup" ticked, the listener auto-starts each time Fusion launches.

**If you accidentally made it a Script** (no "Run on Startup" toggle appears for it, and its path contains `\API\Scripts\`): delete/ignore that one and recreate it under the **Add-Ins** tab as in step 4. The file can't just be moved — add-ins and scripts are registered differently.

---

## Part B — Python and dependencies

1. Have **Python 3.10+** installed and on your PATH. Check:
   `bash python --version # or: python3 --version`
2. Put this package folder somewhere stable, e.g. `C:\Users\<you>\fusion-mcp\` or `~/fusion-mcp/`.
3. Install dependencies **into the interpreter you'll point Claude at**. The safest way is the `-m pip` form so it can't land in the wrong Python: `bash python -m pip install -r requirements.txt` (or `python3 -m pip install -r requirements.txt`)

> **Why `-m pip`?** If you have several Pythons, a bare `pip install` can install into a different one than the launcher/Claude uses, and you'll get "No module named 'requests'" or "No module named 'mcp'" even though pip said success. `python -m pip` installs into *that* python, every time.

---

## Part C — Register with Claude Desktop

1. Open **Claude Desktop → Settings → Developer → Edit Config**. **Always reach the file this way** — it opens the exact file Claude Desktop reads (`claude_desktop_config.json`). Don't go hunting in folders; the location varies and editing the wrong file is the #1 reason "nothing happens."
2. Add a `fusion360` server. If the file is empty, make it exactly:

**Windows:** `json { "mcpServers": { "fusion360": { "command": "python", "args": ["C:/Users/<you>/fusion-mcp/fusion_mcp_server.py"] } } }`

**macOS / Linux:** `json { "mcpServers": { "fusion360": { "command": "python3", "args": ["/Users/<you>/fusion-mcp/fusion_mcp_server.py"] } } }`

- Use an **absolute** path with **forward slashes** (or doubled `\\` on Windows — never single backslashes).
- If the config already has `mcpServers` with other entries, **add `fusion360` as another key inside it** — don't paste a second `mcpServers` block or a second top-level `{ }`. 3. **Fully quit and reopen** Claude Desktop (not just close the window).

---

## Verify

1. With Fusion open (listener running) and Claude Desktop restarted, run the preflight from this folder: - **Windows:** double-click `launch.bat`, or `launch.bat check` - **macOS/Linux:** `./launch.sh` (run `chmod +x launch.sh` once first)

Everything should read **OK**. If `requests` or `mcp` shows FAIL, redo the `python -m pip install` with the right interpreter. 2. In Claude Desktop, confirm the `fusion360` tools appear (tools/connector icon near the chat box), then ask:

> **Ping Fusion.**

A reply of **"alive"** confirms the whole chain: Claude → server → listener. 3. First model:

> **Make a 20 mm cube and show me.**

---

## Is the server running? (a common confusion)

You **don't** start `fusion_mcp_server.py` yourself, and the launcher does **not** keep it running. Here's the model:

- **Claude Desktop launches the server** automatically (as a hidden background child process) whenever it starts, based on the config. It stops when you quit Claude Desktop.
- **The launcher only inspects** your setup — it never leaves anything running. When `launch.bat` says "press any key" and the window closes, nothing was started or stopped.
- **To know it's running:** ask Claude to "Ping Fusion." If the `fusion360` tools show up and the ping works, it's running. If the tools never appear, Claude Desktop couldn't launch it — check the config path and the interpreter.

The only time you'd run the server in a terminal is debugging: `python launch.py serve` (used with the MCP Inspector). Not needed normally.

---

## Common snags

**"No module named 'requests'" (or 'mcp') even after pip installed it.** You installed into a different Python than the launcher/Claude uses. Re-run with `python -m pip install -r requirements.txt` using the *same* command ( `python` vs `python3` ) that your Claude config's `command` uses. The preflight prints which interpreter it's using — match that.

**Preflight says "Claude Desktop config not found."** The launcher checks the standard location ( `%APPDATA%\Claude\claude_desktop_config.json` / `~/Library/Application Support/Claude/...` ).

If yours is elsewhere, that's fine — trust **Settings → Developer → Edit Config** as the source of truth instead. The WARN doesn't block anything.

**Tools don't appear in Claude Desktop.** Almost always the config: invalid JSON (a stray/missing comma), a relative or wrong path, or the wrong interpreter. Validate the JSON, use an absolute path, fully restart Claude Desktop. A quick JSON sanity check:

```
python -m json.tool < "path/to/claude_desktop_config.json"
```

If that errors, your JSON is malformed.

**"Connection error" when a tool runs, but the server is fine.** That's the *listener* — Fusion isn't open, or the add-in isn't running. Shift+S → Add-Ins → MCPListener → Run. Then "Ping Fusion" again.

**Python 3.13 / 3.14.** The SDK generally imports on very new Pythons, but they run ahead of what some dependencies expect. If the server misbehaves in ways the config can't explain, install a 3.11 or 3.12 alongside and point the Claude config `command` at that interpreter's full path. This is the first thing to try for weird, hard-to-place errors.

**OneDrive "doubled" paths.** Some Windows setups have paths like `C:\Users\<you>\Documents\Documents\...` (OneDrive redirection — the doubled `Documents` is real). Whatever your actual path is, make the Claude config match it character-for-character.

**Geometry comes out 10× too big or small.** A units slip somewhere in custom code. All tool arguments are millimeters; Fusion's API is centimeters. Ask Claude to verify with `get_model_state` or `measure_distance`.

**Save / open / list-designs tools error.** Those need you signed into Fusion with an **active project** selected in the Data Panel. They don't work offline or on a brand-new unsaved doc.

# TOOLS.md — Fusion 360 MCP tool reference

**Unit rule:** every argument is in **millimeters**. Internally tools convert to Fusion's centimeters. When you (or Claude) write raw code, the API is in cm.

**Face selection rule:** `top/bottom` = max/min Z, `front/back` = min/max Y, `right/left` = max/min X. Chosen by extreme face-centroid position. Reliable on box-like parts; verify with `view_model` on complex geometry.

---

## Core

### `ping_fusion()`

Liveness + busy check, served on a **separate channel** from geometry execution — so it answers even while a long op runs on the main thread (a hung op no longer blocks ping). Reports `busy` and `current_job`.

### `get_state()`

**Cheap, guaranteed-fast** doc state from the listener cache: active document name, lineage id, and body/sketch/component/timeline counts. Does **no geometry work** and is served without the main thread — safe to call during a heavy op or after a suspected hang. Use first for crash recovery. (For full per-body detail, `get_model_state`.)

### `heartbeat()`

Crash-vs-busy check with explicit recovery instructions. Same liveness data as `ping_fusion`, plus guidance: if it errors, restart the add-in; if `busy=True`, poll `job_status`.

### `run_async(code)`

Start a long operation **without waiting** — returns a `job_id` immediately. Poll `job_status(job_id)` for incremental progress and the final result. Use for anything expected to exceed ~90s (large patterns, heavy booleans, full-assembly rebuilds). Same injected helpers and unit rules. Call `progress(msg)` in your code to mark stages visible while it runs.

### `job_status(job_id)`

Structured result of an async job: `status` (queued/running/ok/error/timeout), `elapsed_ms`, stdout collected **so far** (works mid-run), and traceback on failure. Poll until status is ok/error/timeout.

### `run_fusion_code(code)`

Execute arbitrary Python in Fusion via the **bounded sync path**: waits up to ~100s and returns a structured envelope (status / elapsed / stdout / traceback). If the job outlives the wait it keeps running and stays pollable — the response tells you the job id. `adsk`, `app`, `ui` in scope, plus injected helpers (below). **Use `mm(x)` for every length** instead of hand-dividing by 10.

The docstring also documents the **sketch-plane axis mapping** (XY/XZ/YZ and offset planes) so the XZ→world-−Z inversion doesn't cost you a debugging cycle.

**Injected helpers** (available in `run_fusion_code` and `run_fusion_code_safe`, no import needed): - `mm(x)` / `cm(x)` — mm→cm for values you pass to the API. - `to_mm(x)` / `to_mm2(a)` / `to_mm3(v)` — cm→mm for reporting. - `assert_scale(max_mm=2000)` — raise if any body is 10×-too-big (units slip). - `total_volume()` — summed cm³ of all bodies; capture **before** a boolean. - `report_volume_delta(before, label)` — print the change and **scream if ~0** (silent no-op boolean). - `sliver_face_count(body)` — count sub-threshold faces (bad-boolean slivers). - `log_sketch_areas(sketch)` — print each profile area in mm² (catches scale slips). - `delete_face(face)` — delete one face the way Fusion accepts ( `DeleteFaceFeatures` ).

Recommended pattern for any cut/join in raw code:

```
_b = total_volume()
# ... do the boolean ...
report_volume_delta(_b, 'mycut'); assert_scale()
```

**Known-unstable (refused by the listener):** move-face via `defineAsFreeMove` (hard-crashes the host) and `defineAsOffset` (missing on this build). Use parametric edits / `offsetFeatures` / shell instead.

### `run_fusion_code_safe(code)`

Transactional `run_fusion_code`: records the timeline marker and, on error, rolls back and deletes features created after it. Same injected helpers.

### `clear_document()`

Delete all bodies, sketches, and timeline features. Returns design to empty.

---

## Parameters

`set_parameter(name, value_mm)`

Create or update a user parameter (mm). Build parametric-first for anything that might change later.

`list_parameters()`

List user parameters and their expressions.

---

## Primitives (root component, XY plane)

> **Auto-observability:** the material-removing tools ( `drill_hole` , `hole_on_face` , `tapped_hole` , `shell_body` ) automatically report the **volume delta** of the operation and **warn if it's ~zero** (a silent no-op cut that didn't touch the body). `shell_body` also reports sliver-face count. You no longer need to probe volume yourself to confirm a cut worked.

`create_box(length_mm, width_mm, height_mm, origin_x_mm=0, origin_y_mm=0)`

Rectangle on XY extruded up by height. Returns new body name.

`create_cylinder(diameter_mm, height_mm, origin_x_mm=0, origin_y_mm=0)`

Circle on XY extruded up.

`drill_hole(diameter_mm, x_mm, y_mm, depth_mm, body_index=0)`

Cut a hole straight down from XY at (x, y) to depth.

`fillet_edges(radius_mm, body_index=0, which="all")`

Round edges. `which` : `all` / `vertical` (vertical-running edges) / `top` (edges lying on the top face plane). Warns if nothing matched.

`shell_body(thickness_mm, body_index=0, open_top=True)`

Hollow to a wall thickness. `open_top=True` removes the top face (open box); `False` makes a closed hollow body.

`tapped_hole(thread_size, x_mm, y_mm, depth_mm, body_index=0, tap_drill_only=True)`

Tapped hole on the top (XY) plane, cutting down. `thread_size` e.g. `M3`, `M3x0.5`, `M4`, `M5`, `M6`, `1/4-20`, `10-32`, `#6-32`. - `tap_drill_only=True` (default): cuts only the correctly-sized tap-drill clearance hole — best for 3D-printed and hand-tapped parts. - `False`: also applies a **modeled** thread (visualization / CNC); best-effort, may be skipped with a warning if Fusion can't match the designation. Known tap-drill sizes include metric M1.6–M12 and common imperial; unknown sizes fall back to M3's drill — pass an explicit size you know is in the table, or ask to extend it.

---

## Face-referenced features

`sketch_on_face(component_name="", which="top", body_index=0)`

Select a face by orientation and stash it. Empty `component_name` = root. Call `hole_on_face` immediately after (single-slot; don't interleave).

`hole_on_face(diameter_mm, u_mm, v_mm, depth_mm, through=False)`

Drill into the stashed face. `(u, v)` are mm offsets from the face centroid along the face's own axes. `through=True` cuts all the way.

---

## Measurement & state

`get_model_state()`

Comprehensive design state in one call. Per body: name, validity, volume (cm³), bbox (mm), face/edge counts, and **sliver-face count** (flags a bad boolean). Plus body/sketch counts and the **timeline feature names**. Replaces hand-rolled introspection; makes crash recovery a one-call operation.

`measure_distance(which_a, which_b, body_index=0)`

True centroid-to-centroid distance (mm) between two oriented faces. Use to verify wall thickness, overall size, gaps — more trustworthy than the bbox.

`cross_section(body_index=0, axis="z", at_mm=None, span_axis="x", samples=41)`

Scan a line of points through a body and report solid/void as an **ASCII row** (`#`=solid, `.`=void) plus the **mm coordinates of every solid/void boundary**. The best non-visual diagnostic for trapped voids, missed cuts, and wall thickness. Holds the two non-span axes at their mid-value (or at `at_mm` along `axis`) and sweeps `span_axis` across the bbox. Example: `cross_section(axis="z", at_mm=10, span_axis="x")` → reads off wall positions along X at Z=10 mm.

`list_faces(body_index=0, max_faces=40)`

Faces with a stable **tempId**, centroid (mm), area (mm²), surface type, largest first. The tempId lets you re-select the same face later via `body.findByTempId(tempId)` in raw code — essential for multi-step face work.

`list_edges(body_index=0, max_edges=40)`

Edges with a stable **tempId**, length (mm), midpoint (mm), longest first. Use the tempId to target specific edges for fillets/chamfers in raw code.

---

## Visual feedback

`view_model(orientation="iso")`

Orient (iso/top/front/back/right/left), fit, screenshot, and **return the image** so Claude sees it. Call after non-trivial operations to verify.

`set_view(orientation="iso")`

Reposition the camera without returning an image.

---

## Components & occurrences

`create_component(name)`

New empty component as an occurrence at the origin. One per distinct part.

`list_components()`

All occurrences: fullPathName, component name, origin position (mm).

`box_in_component(component_name, length_mm, width_mm, height_mm, ox_mm=0, oy_mm=0, oz_mm=0)`

Box built inside a named component on its own XY plane.

`move_component(component_name, x_mm, y_mm, z_mm)`

Translate an occurrence's origin. Position BEFORE jointing.

`copy_component(component_name, count, dx_mm=0, dy_mm=0, dz_mm=0)`

Place `count` more occurrences of one definition in a linear pattern. Reuses the definition (edit once, all update) — ideal for repeated hardware.

`ground_component(component_name, grounded=True)`

Lock/unlock a component in space. Ground exactly one base per assembly, first.

---

## Joints

Joints reference component **origins** (or explicit points). To pivot at a specific location, design the component so its local origin sits there, or use `joint_at_point`. Position parts first; joints override transforms after.

`rigid_joint(component_a, component_b)`

Lock two components rigidly at their current relative position.

`revolute_joint(component_a, component_b, axis="z")`

One rotational DOF about `axis` (x/y/z). Hinges, lids, arms.

`joint_at_point(component_a, ax_mm, ay_mm, az_mm, component_b, bx_mm, by_mm, bz_mm, motion="rigid", axis="z")`

Joint at explicit local points on each component. `motion`: rigid/revolute.

`list_joints()`

All joints and their motion type.

---

## Export

`export_step(filepath)`

STEP of the whole design. Absolute path, `.step`.

`export_stl(filepath, body_index=0, quality="medium")`

STL of one body for 3D printing. `quality`: low/medium/high.

`export_dxf(filepath, sketch_index=0, component_name="")`

DXF of a sketch profile for laser/waterjet/flat stock. Selects sketch by index in root (or a named component).

---

## Document persistence

Require being signed in with an **active project** in Fusion's Data Panel.

> **Document identity is now surfaced everywhere.** `get_state` and `get_model_state` both lead with the active document's **name + lineage id**, so you can see which document a `clear` / `save` / `open` will affect before acting. This prevents the worst error class — overwriting the wrong design.

`save_document(name="", create_new=False)`

Save the active doc and **always report which lineage was written**. - `create_new=True` → save-as a brand-new document/lineage named `name`. Use for a **new part** so it never lands in an unrelated design's history. - `create_new=False` (default) → unsaved doc saves-as `name`; already-saved doc saves a **new version of the current lineage**, printing the name + lineage id it versioned. **Warning:** this versions whatever lineage is open — if you opened part A but meant to save new part B, it buries B in A's history. When in doubt, `create_new=True` or check identity first.

`list_recent_designs(limit=15)`

Designs in the active project root, as `name :: id`.

`list_versions(design_id="")`

**Read-only:** enumerate the saved versions of a lineage (version number, id, date) so you can confirm what's recoverable after a bad save and point the user to the exact version to promote in the Data Panel. Empty `design_id` = active document's lineage.

`open_document(design_id)`

Open a design by id. **Changes the active document** — subsequent `clear` / `save` act on it. Reports the new lineage; confirm with `get_state()`.

---

## Typical workflows

**Single part:** `clear_document` → `set_parameter`(s) → primitive(s) → features ( `fillet_edges` , `shell_body` , `tapped_hole` , `sketch_on_face` + `hole_on_face` ) → `view_model` → `get_model_state` / `measure_distance` → `export_stl` / `export_dxf` .

**Assembly:** `create_component` per part → `box_in_component` (or build geometry) → `ground_component` base → `move_component` others → `rigid_joint` / `revolute_joint` / `joint_at_point` → `view_model` → `save_document` .

**Resume later:** `list_recent_designs` → `open_document(id)` → modify → `save_document` (new version).

---

## Known-unstable operations (refused by the listener)

The listener actively **refuses** these calls before they reach Fusion, because they have hard-crashed the host in practice (which kills the listener and requires a manual add-in restart). If you send code containing them, you'll get a `REFUSED (stability guard)` message instead of a crash:

| Call | Why refused | Use instead |
|---|---|---|
| `*.defineAsFreeMove(...)` (move-face free move) | Hard-crashes the Fusion host | Edit the parametric feature, or rebuild the body |
| `*.defineAsOffset(...)` | Not reliably available on this build | `offsetFeatures` , or a shell / offset-plane extrude |

If you find another operation that reliably crashes the host, add its call signature to `_DANGEROUS_PATTERNS` in `MCPListener.py` and document it here.

## Diagnosing a hang or suspected crash

The listener runs jobs **asynchronously** and answers ping/state on a separate channel, so you can always tell what's happening:

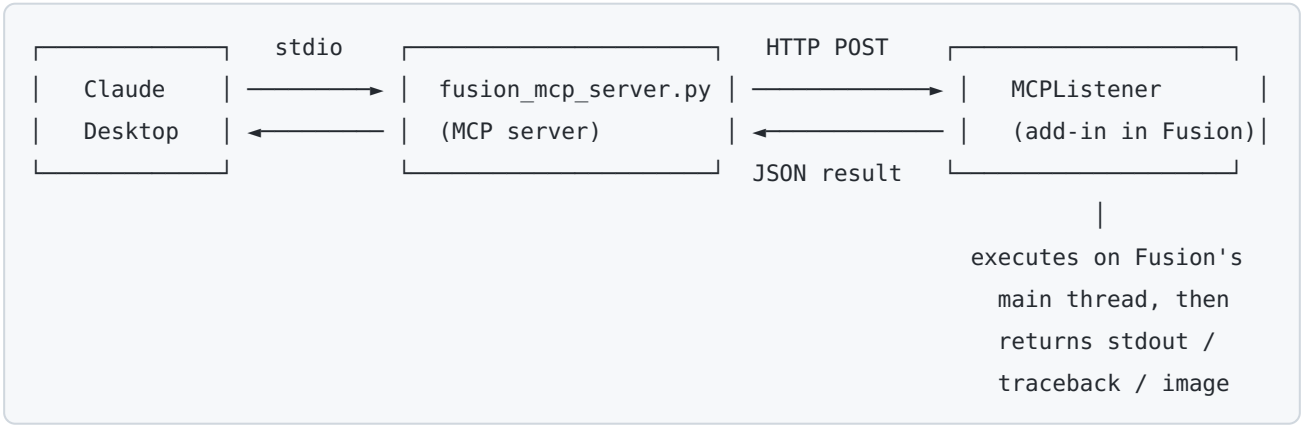
1. `heartbeat()` (or `ping_fusion()` ). If it responds with `busy=True` , a job is still running — `job_status(current_job)` for its partial output. If it errors/times out, the listener is down.
2. If a `run_fusion_code` call returned "STILL RUNNING" or a timeout envelope, the job kept going — poll `job_status(job_id)` rather than re-issuing it (re-issuing stacks another job behind the stuck one).
3. `get_state()` is always cheap and never blocks — use it to see the active document and counts even mid-operation.
4. If the listener is truly down: in Fusion, **Shift+S → Add-Ins → MCPListener → Run** to restart it, then `get_state()` to see what survived.

A job that exceeds its budget is flagged `timeout` in the envelope (the watchdog can't force-kill a hung main thread, but you get a clear signal instead of a dead channel).

# Architecture

For the curious, and for anyone wanting to extend the toolset.

## The processes




Three participants:

1. **Claude Desktop** reads `claude_desktop_config.json`, launches the MCP server as a child process, and speaks the MCP protocol to it over stdio.
2. `fusion_mcp_server.py` is a FastMCP server. Each tool builds a small Python snippet (or passes user code through) and HTTP-POSTs it as JSON to the listener, returning the listener's response to Claude.
3. `MCPListener` is a Fusion add-in. It runs an `http.server` on `127.0.0.1:8765` with async endpoints: `POST /exec` (enqueue, returns job id), `GET /job/<id>` (envelope), `GET /ping` and `GET /state` (served without the main thread), and `POST /exec_sync` (bounded blocking, back-compat). Incoming code runs on the main thread; status, stdout, and tracebacks go into a job store.

## Why two processes instead of one?

Fusion's Python API can only be called **from inside Fusion's own interpreter**, and only on its **main thread**. An external MCP server can't touch the API directly. So the add-in lives inside Fusion to hold API access, and the MCP server lives outside (where Claude Desktop can spawn it) and talks to the add-in over localhost HTTP.

## The async job model and the main-thread handoff

An HTTP server needs a background thread to listen, but Fusion's API rejects calls from any thread except its main one. And a *hung* geometry call blocks that main thread — so no HTTP cleverness can make a queued geometry call jump ahead. The design works around this:

**Jobs are async.** `POST /exec` enqueues a job, fires a Fusion custom event (`MCPListenerExec`) with the job id, and returns the id **immediately**. The event is dispatched on the main thread, where a handler runs the code and writes status + stdout into a shared job store. `GET /job/<id>` returns a structured envelope (`status`, `stdout`, `traceback`, `elapsed_ms`) — and reads partial stdout while the job is still running (incremental progress via a live buffer).

**Ping and state never touch the main thread.** `GET /ping` and `GET /state` are answered entirely from cached data on the HTTP thread. So even while a geometry call churns (or hangs) on the main thread, liveness and cheap document state stay responsive. This is what makes "is it busy or dead?" answerable.

**A watchdog flags overruns.** A background thread marks any job that exceeds its budget as `status: timeout`. It can't force-kill a hung main thread (Fusion's API has no such hook), but the caller gets an immediate, unambiguous signal instead of a dead socket.

**Sync path for convenience.** `POST /exec_sync` enqueues then waits up to `wait_s` and returns the envelope — used by `run_fusion_code` for short ops. If the op outlives the wait, it keeps running and stays pollable by job id; the response says so and returns partial stdout. The server's HTTP client timeout sits above `wait_s`, so the agent always gets a structured envelope, never a bare socket timeout.

## Units

Fusion's API is internally in **centimeters**. Every tool exposes **millimeters** to Claude and divides by 10 before talking to the API. When writing raw code via `run_fusion_code`, you're in cm — divide mm by 10 yourself.

## The safety wrapper

`run_fusion_code_safe` records the timeline marker position, runs your code, and on exception rolls the timeline back and deletes any features created after the marker — leaving the document as it was. It's effective on parametric documents (design history on, the default). Direct-modeling docs have no timeline to roll back, so it degrades to plain error reporting.

## Visual feedback

`view_model` orients the camera, fits, saves a PNG via `Viewport.saveAsImageFile`, reads the bytes back, and returns them as an MCP `Image`. Because the MCP server and Fusion are on the same machine, the file round-trips through `~/fusion_view.png`. (This is the one tool that assumes shared local disk.)

## Security model

- The listener binds to **`127.0.0.1` only** — not reachable off-machine.
- It does, by design, **execute arbitrary Python** it receives. That's the whole mechanism. The trust boundary is: only your local MCP server (driven by your Claude Desktop) talks to it.
- **Do not** change the bind address to `0.0.0.0` or forward the port. Doing so would let anything on your network run code in your Fusion session.
- There's no authentication on the localhost channel; it relies on the OS loopback boundary. Fine for a personal machine, not for a shared/multi-user host.

## Extending the toolset

Add a new tool by writing a function in `fusion_mcp_server.py` decorated with `@mcp.tool()`. Build a Fusion API snippet as a string, `print()` whatever you want Claude to see, and return `_send(code)`. Patterns to copy:

- **Take mm, convert to cm** at the top ( `x = x_mm / 10.0` ).
- **`print()` a clear success line** — that string is what Claude reads back.
- **Guard lookups** (component/body not found) and print an `ERROR: ...` line rather than throwing, so Claude gets a readable message.
- For multi-step features, consider routing through the same rollback idea as `run_fusion_code_safe`.

Restart Claude Desktop after editing the server so it reloads the tool list.

## Files

| File | Runs where | Role |
|---|---|---|
| `MCPListener.py` | Inside Fusion | HTTP listener + main-thread executor |
| `fusion_mcp_server.py` | Spawned by Claude Desktop | MCP tools → HTTP relay |
| `launch.py` | Your terminal | Preflight/health checks only |

# Fusion CAD — Claude Desktop project instructions

## Paste everything below the line into the custom instructions of a Claude

## Desktop Project, and do your Fusion modeling chats inside that project.

---

You can drive Fusion 360 through MCP tools (server name: fusion360). Follow these conventions.

UNITS - All tool inputs are in MILLIMETERS. Tools convert to Fusion's internal cm. - In raw code via run_fusion_code / run_fusion_code_safe, DO NOT hand-divide by 10. Use the injected helper mm(x) for every length you pass to the API. This is the #1 source of 10x-scale bugs. Call assert_scale() after building to catch any slip immediately.

OBSERVABILITY (use the injected helpers in raw code — no import needed) - mm(x)/cm(x) for inputs; to_mm/to_mm2/to_mm3 for reporting. - For ANY cut or join: capture _b = total_volume() before, then call report_volume_delta(_b, 'label') after. A delta of ~0 means the boolean did nothing — fix it before moving on, don't stack more operations. - Use sliver_face_count(body) after booleans to catch bad geometry. - Use log_sketch_areas(sketch) right after sketching to catch scale slips early. - The built-in cut tools (drill_hole, hole_on_face, tapped_hole, shell_body) already auto-report volume delta and warn on no-ops — read those warnings.

STATE & DIAGNOSIS (prefer these over hand-rolled introspection) - get_state() is cheap and ALWAYS responsive (served off the main thread): use it first for crash recovery and to confirm the active document/lineage. - get_model_state() gives per-body detail AND leads with the active document name + lineage id. Check that identity before any clear/save. - cross_section(...) prints an ASCII solid/void scan

+ boundary coordinates: the fastest way to verify wall thickness, trapped voids, or a missed cut. - list_faces()/list_edges() return stable tempIds for re-selecting entities.

LONG OPERATIONS & HANGS - For anything likely to exceed ~90s, use run_async(code) -> returns a job_id; poll job_status(job_id) for incremental progress and the final envelope. Emit progress(msg) in your code to mark stages. - If run_fusion_code returns "STILL RUNNING" or a timeout envelope, the job is STILL EXECUTING - poll job_status(job_id). Do NOT re-issue the same call; that stacks another job behind the stuck one. - ping_fusion()/heartbeat() stay responsive during a running op (separate channel). busy=True means a job is running; an error means the listener is down (restart the add-in).

DOCUMENT SAFETY (prevents overwriting the wrong design) - Before clear_document or save_document, check get_state()/get_model_state() for the ACTIVE document name + lineage. open_document changes which document is active. - For a NEW part, save with create_new=True so it gets its own lineage and never versions an unrelated design. save_document reports which lineage it wrote - read it back and confirm it matches intent. - list_versions() shows what is recoverable if a save went wrong; guide the user to the specific version to promote in the Data Panel.

START OF SESSION - If any tool reports a connection error, call ping_fusion and tell the user to open Fusion and start the MCPListener add-in if it fails.

WORKFLOW PER PART 1. Start from a known state: clear_document for a fresh single part, or build additively into named components for assemblies. 2. Build parametric-first. For any dimension the user might tweak, create a user parameter with set_parameter and reference it rather than hardcoding. 3. After any non-trivial operation, call view_model to SEE the result. Don't trust a success message alone. Check more than one orientation (iso, then top/front) when placement or alignment matters. 4. Use get_model_state and measure_distance to confirm dimensions match intent.

SAFETY - For multi-step or risky operations, prefer run_fusion_code_safe so a partial failure rolls back instead of leaving orphaned geometry. - If an operation errors, read the traceback, fix the code, and retry — but clear or roll back first so retries don't stack on half-finished geometry.

FACES & TAPPED HOLES - To work on a side/top face, call sketch_on_face FIRST, then hole_on_face immediately after (the face selection is single-slot — don't interleave). - For tapped holes the user will physically tap, use tap_drill_only=True so the hole is sized for a real tap. Reserve modeled threads for CNC/visualization.

ASSEMBLIES - One component per distinct part. Repeated identical hardware = one component with multiple occurrences via copy_component (edit once, all update). - Ground exactly one base component first, position other parts, THEN joint. - Joints win over transforms: don't move a part after jointing it.

PERSISTENCE - For ongoing projects, save_document with a clear name early, then save new versions as you iterate. Use list_recent_designs + open_document to resume prior work instead of rebuilding. Confirm the intended Fusion project is active before a named save.

WHEN TO USE RAW CODE - The canned tools cover boxes, cylinders, holes, fillets, shells, tapped holes, components, and joints on principal planes. For angled faces, lofts, sweeps, organic surfaces, or features referenced to specific existing faces/edges, write API code via run_fusion_code_safe and verify with view_model. - NEVER use move-face via defineAsFreeMove (it hard-crashes the host) or

defineAsOffset (missing on this build). The listener refuses both. Use parametric feature edits, offsetFeatures, or a rebuild instead.

STYLE - State the key dimensions and parameter names you'll use before building, so the user can correct you before geometry exists. - Keep the user informed which body/component you're operating on.

# Publishing to GitHub

Quick steps to put this online so friends can clone it.

## One-time setup

From inside the `fusion-mcp/` folder:

```
git init
git add .
git commit -m "Initial release: Fusion 360 MCP v1.0.0"
```

Create an empty repo on GitHub (no README/license — this package has its own), then:

```
git branch -M main
git remote add origin https://github.com/<your-username>/fusion-mcp.git
git push -u origin main
```

## Before you push — check for personal paths

The code itself contains no personal paths, but double-check you haven't pasted your real `claude_desktop_config.json` (with your username) into the repo. The `.gitignore` already excludes `fusion_view.png`, `__pycache__`, and `claude_desktop_config.local.json`. The config lives in Claude Desktop's own directory, not here, so it won't be committed unless you copy it in.

## Suggested repo description

> Drive Autodesk Fusion 360 from Claude via MCP — parametric parts, assemblies, joints, tapped holes, export, and a visual feedback loop.

## Suggested topics/tags

`fusion360` `mcp` `model-context-protocol` `claude` `cad` `autodesk` `automation` `parametric-design`

## A note for contributors

If people send PRs, the easiest things to review are new `@mcp.tool()` functions that follow the existing patterns (mm in, cm to the API, print a clear result, guard lookups). See `docs/ARCHITECTURE.md` → "Extending the toolset."

## Releases

Tag versions so people can grab a stable zip:

```
git tag -a v1.0.0 -m "v1.0.0"
git push origin v1.0.0
```

GitHub will auto-generate a downloadable archive for each tag under "Releases."